T3

October 26, 2026

1 Study Project: Adversarial Machine Learning

1.1 Part I: Statistical Analysis and Generation of Process Data

1.2 Group 15

1.3 SIN Wang Yat

1.3.1 Task 3: Generate of Synthetic Dataset

Inputs & Value Descriptions

- Input 1 (Dataset Selection): Choose WADI_14days.csv or WADI_attackdata.csv
- Input 2 (Row Sub-sampling Percentage in 1-100%): Randomly select a fraction of sample rows from the chosen dataset
- Input 3 (Normalization Type Selection): Choose StandardScaler or MinMaxScaler
- Input 4 (Synthetic Generation Percentage in 1-1000%): Indicate the amount of synthetic samples to generate
- Input 5 (Number of Nearest Neighbors k in 2-5): Specify the number of neighbors used for Manhattan distanc

Output Data & CSV Dataset

- Output Data is automatically exported and saved as synthetic_data.csv under the target directory Path.home() /Projects/P1/data/processed/

1.3.2 Libray Import and Data Loading

[1]:
```
print(f"Start: Import Library and Load the datasets")
# Library Import
import pandas as pd
import numpy as np
from pathlib import Path
import random
from datetime import datetime, timedelta

# Define the base directory path
base_path = Path.home() / "Projects/P1/data/"
cache_train_file = base_path / "processed/cache_train_file.parquet"
cache_test_file = base_path / "processed/cache_test_file.parquet"
```

```
# Load normal 14days data and attack data from CSV or Cache
# Load 14days Dataset
if cache_train_file.exists():
   print("Loading fast cached Parquet data-14days dataset")
   train17_df = pd.read_parquet(cache_train_file)
else:
   print("Cache not found. Load data from CSV file-14days dataset")
   # Skip 4 meta data rows
   train17_df = pd.read_csv(base_path / "raw/2017/WADI_14days.csv", skiprows=4)
   # Save the cache for next time
   train17_df.to_parquet(cache_train_file)

# Load attackdata Dataset
if cache_test_file.exists():
   print("Loading fast cached Parquet data-attackdata dataset")
   test17_df = pd.read_parquet(cache_test_file)
else:
   print("Cache not found. Load data from CSV file-attackdata dataset")
   test17_df = pd.read_csv(base_path / "raw/2017/WADI_attackdata.csv")
   # Save the cache for next time
   test17_df.to_parquet(cache_test_file)

# Remove the long prefix Column Name
prefix_to_remove = r"\\WIN-25J4RO10SBF\LOG_DATA\SUTD_WADI\LOG_DATA"

# Replace the string in all column names
train17_df.columns = train17_df.columns.str.replace(prefix_to_remove, "",␣
 ↪regex=False)
test17_df.columns = test17_df.columns.str.replace(prefix_to_remove, "",␣
 ↪regex=False)
```

```
Start: Import Library and Load the datasets
Loading fast cached Parquet data-14days dataset
Loading fast cached Parquet data-attackdata dataset
```

[2]:
```
print(f"Data Shape of WADI_14days")
print(train17_df.shape)
```

```
Data Shape of WADI_14days
(1209601, 130)
```

[3]:
```
print(f"Data info of WADI_14days")
print(train17_df.info(verbose=True,show_counts=True))
print(train17_df)
```

```
Data info of WADI_14days
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 1209601 entries, 0 to 1209600
Data columns (total 130 columns):
 #    Column                      Non-Null Count    Dtype
---   ------                      --------------    -----
 0    Row                         1209601 non-null  int64
 1    Date                        1209601 non-null  object
 2    Time                        1209601 non-null  object
 3    \1_AIT_001_PV               1209601 non-null  float64
 4    \1_AIT_002_PV               1209589 non-null  float64
 5    \1_AIT_003_PV               1209601 non-null  float64
 6    \1_AIT_004_PV               1209595 non-null  float64
 7    \1_AIT_005_PV               1209601 non-null  float64
 8    \1_FIT_001_PV               1209601 non-null  float64
 9    \1_LS_001_AL                1209601 non-null  int64
 10   \1_LS_002_AL                1209601 non-null  int64
 11   \1_LT_001_PV                1209601 non-null  float64
 12   \1_MV_001_STATUS            1209601 non-null  int64
 13   \1_MV_002_STATUS            1209601 non-null  int64
 14   \1_MV_003_STATUS            1209601 non-null  int64
 15   \1_MV_004_STATUS            1209601 non-null  int64
 16   \1_P_001_STATUS             1209601 non-null  int64
 17   \1_P_002_STATUS             1209601 non-null  int64
 18   \1_P_003_STATUS             1209601 non-null  int64
 19   \1_P_004_STATUS             1209601 non-null  int64
 20   \1_P_005_STATUS             1209601 non-null  int64
 21   \1_P_006_STATUS             1209601 non-null  int64
 22   \2_DPIT_001_PV              1209601 non-null  float64
 23   \2_FIC_101_CO               1209601 non-null  float64
 24   \2_FIC_101_PV               1209601 non-null  float64
 25   \2_FIC_101_SP               1209601 non-null  float64
 26   \2_FIC_201_CO               1209601 non-null  float64
 27   \2_FIC_201_PV               1209601 non-null  float64
 28   \2_FIC_201_SP               1209601 non-null  float64
 29   \2_FIC_301_CO               1209601 non-null  float64
 30   \2_FIC_301_PV               1209601 non-null  float64
 31   \2_FIC_301_SP               1209601 non-null  float64
 32   \2_FIC_401_CO               1209601 non-null  float64
 33   \2_FIC_401_PV               1209601 non-null  float64
 34   \2_FIC_401_SP               1209601 non-null  float64
 35   \2_FIC_501_CO               1209601 non-null  float64
 36   \2_FIC_501_PV               1209601 non-null  float64
 37   \2_FIC_501_SP               1209601 non-null  float64
 38   \2_FIC_601_CO               1209601 non-null  float64
 39   \2_FIC_601_PV               1209601 non-null  float64
 40   \2_FIC_601_SP               1209601 non-null  float64
 41   \2_FIT_001_PV               1209601 non-null  float64
 42   \2_FIT_002_PV               1209601 non-null  float64
 43   \2_FIT_003_PV               1209601 non-null  float64
```

```
44   \2_FQ_101_PV             1209601 non-null  float64
45   \2_FQ_201_PV             1209601 non-null  float64
46   \2_FQ_301_PV             1209601 non-null  float64
47   \2_FQ_401_PV             1209601 non-null  float64
48   \2_FQ_501_PV             1209601 non-null  float64
49   \2_FQ_601_PV             1209601 non-null  float64
50   \2_LS_001_AL             0 non-null        float64
51   \2_LS_002_AL             0 non-null        float64
52   \2_LS_101_AH             1209601 non-null  int64
53   \2_LS_101_AL             1209601 non-null  int64
54   \2_LS_201_AH             1209601 non-null  int64
55   \2_LS_201_AL             1209601 non-null  int64
56   \2_LS_301_AH             1209601 non-null  int64
57   \2_LS_301_AL             1209601 non-null  int64
58   \2_LS_401_AH             1209601 non-null  int64
59   \2_LS_401_AL             1209601 non-null  int64
60   \2_LS_501_AH             1209601 non-null  int64
61   \2_LS_501_AL             1209601 non-null  int64
62   \2_LS_601_AH             1209601 non-null  int64
63   \2_LS_601_AL             1209601 non-null  int64
64   \2_LT_001_PV             1209601 non-null  float64
65   \2_LT_002_PV             1209601 non-null  float64
66   \2_MCV_007_CO            1209601 non-null  int64
67   \2_MCV_101_CO            1209601 non-null  float64
68   \2_MCV_201_CO            1209601 non-null  float64
69   \2_MCV_301_CO            1209601 non-null  float64
70   \2_MCV_401_CO            1209601 non-null  float64
71   \2_MCV_501_CO            1209601 non-null  float64
72   \2_MCV_601_CO            1209601 non-null  float64
73   \2_MV_001_STATUS         1209601 non-null  int64
74   \2_MV_002_STATUS         1209601 non-null  int64
75   \2_MV_003_STATUS         1209601 non-null  int64
76   \2_MV_004_STATUS         1209601 non-null  int64
77   \2_MV_005_STATUS         1209601 non-null  int64
78   \2_MV_006_STATUS         1209601 non-null  int64
79   \2_MV_009_STATUS         1209601 non-null  int64
80   \2_MV_101_STATUS         1209601 non-null  int64
81   \2_MV_201_STATUS         1209601 non-null  int64
82   \2_MV_301_STATUS         1209601 non-null  int64
83   \2_MV_401_STATUS         1209601 non-null  int64
84   \2_MV_501_STATUS         1209601 non-null  int64
85   \2_MV_601_STATUS         1209601 non-null  int64
86   \2_P_001_STATUS          0 non-null        float64
87   \2_P_002_STATUS          0 non-null        float64
88   \2_P_003_SPEED           1209601 non-null  float64
89   \2_P_003_STATUS          1209601 non-null  int64
90   \2_P_004_SPEED           1209601 non-null  float64
91   \2_P_004_STATUS          1209601 non-null  int64
```

```
 92   \2_PIC_003_CO              1209601 non-null  float64
 93   \2_PIC_003_PV              1209601 non-null  float64
 94   \2_PIC_003_SP              1209601 non-null  int64
 95   \2_PIT_001_PV              1209601 non-null  float64
 96   \2_PIT_002_PV              1209601 non-null  float64
 97   \2_PIT_003_PV              1209601 non-null  float64
 98   \2_SV_101_STATUS           1209601 non-null  int64
 99   \2_SV_201_STATUS           1209601 non-null  int64
 100  \2_SV_301_STATUS           1209601 non-null  int64
 101  \2_SV_401_STATUS           1209601 non-null  int64
 102  \2_SV_501_STATUS           1209601 non-null  int64
 103  \2_SV_601_STATUS           1209601 non-null  int64
 104  \2A_AIT_001_PV             1209601 non-null  float64
 105  \2A_AIT_002_PV             1209601 non-null  float64
 106  \2A_AIT_003_PV             1209601 non-null  float64
 107  \2A_AIT_004_PV             1209601 non-null  float64
 108  \2B_AIT_001_PV             1209601 non-null  float64
 109  \2B_AIT_002_PV             1209601 non-null  float64
 110  \2B_AIT_003_PV             1209601 non-null  float64
 111  \2B_AIT_004_PV             1209581 non-null  float64
 112  \3_AIT_001_PV              1209601 non-null  float64
 113  \3_AIT_002_PV              1209595 non-null  float64
 114  \3_AIT_003_PV              1209601 non-null  float64
 115  \3_AIT_004_PV              1209595 non-null  float64
 116  \3_AIT_005_PV              1209601 non-null  float64
 117  \3_FIT_001_PV              1209601 non-null  float64
 118  \3_LS_001_AL               1209601 non-null  int64
 119  \3_LT_001_PV               1209601 non-null  float64
 120  \3_MV_001_STATUS           1209601 non-null  int64
 121  \3_MV_002_STATUS           1209601 non-null  int64
 122  \3_MV_003_STATUS           1209601 non-null  int64
 123  \3_P_001_STATUS            1209601 non-null  int64
 124  \3_P_002_STATUS            1209601 non-null  int64
 125  \3_P_003_STATUS            1209601 non-null  int64
 126  \3_P_004_STATUS            1209601 non-null  int64
 127  \LEAK_DIFF_PRESSURE        1209601 non-null  float64
 128  \PLANT_START_STOP_LOG      1209601 non-null  int64
 129  \TOTAL_CONS_REQUIRED_FLOW  1209601 non-null  float64
dtypes: float64(71), int64(57), object(2)
memory usage: 1.2+ GB
None
             Row       Date            Time  \1_AIT_001_PV  \1_AIT_002_PV  \
0              1  9/25/2017  6:00:00.000 PM        171.155       0.619473
1              2  9/25/2017  6:00:01.000 PM        171.155       0.619473
2              3  9/25/2017  6:00:02.000 PM        171.155       0.619473
3              4  9/25/2017  6:00:03.000 PM        171.155       0.607477
4              5  9/25/2017  6:00:04.000 PM        171.155       0.607477
…           …         …              …             …             …
```

```
1209596  1209597  10/9/2017  5:59:56.000 PM        164.211       0.511488
1209597  1209598  10/9/2017  5:59:57.000 PM        164.211       0.511488
1209598  1209599  10/9/2017  5:59:58.000 PM        164.211       0.511488
1209599  1209600  10/9/2017  5:59:59.000 PM        164.211       0.511488
1209600  1209601  10/9/2017  6:00:00.000 PM        164.210       0.529486

         \1_AIT_003_PV  \1_AIT_004_PV  \1_AIT_005_PV  \1_FIT_001_PV  \
0              11.5759        504.645       0.318319       0.001157
1              11.5759        504.645       0.318319       0.001157
2              11.5759        504.645       0.318319       0.001157
3              11.5725        504.673       0.318438       0.001207
4              11.5725        504.673       0.318438       0.001207
…                …              …             …              …
1209596        11.9928        482.452       0.331198       0.001182
1209597        11.9928        482.452       0.331198       0.001182
1209598        11.9928        482.452       0.331198       0.001182
1209599        11.9928        482.452       0.331198       0.001182
1209600        11.9972        482.480       0.331167       0.001273

         \1_LS_001_AL  …  \3_MV_001_STATUS  \3_MV_002_STATUS  \
0                   0  …                 1                 1
1                   0  …                 1                 1
2                   0  …                 1                 1
3                   0  …                 1                 1
4                   0  …                 1                 1
…                 …  …                 …                 …
1209596             0  …                 1                 1
1209597             0  …                 1                 1
1209598             0  …                 1                 1
1209599             0  …                 1                 1
1209600             0  …                 1                 1

         \3_MV_003_STATUS  \3_P_001_STATUS  \3_P_002_STATUS  \3_P_003_STATUS  \
0                       1                 1                 1                 1
1                       1                 1                 1                 1
2                       1                 1                 1                 1
3                       1                 1                 1                 1
4                       1                 1                 1                 1
…                     …                 …                 …                 …
1209596                 1                 1                 1                 1
1209597                 1                 1                 1                 1
1209598                 1                 1                 1                 1
1209599                 1                 1                 1                 1
1209600                 1                 1                 1                 1

         \3_P_004_STATUS  \LEAK_DIFF_PRESSURE  \PLANT_START_STOP_LOG  \
0                       1              67.9651                      1
1                       1              67.9651                      1
```

```
2                    1              67.9651                    1
3                    1              67.1948                    1
4                    1              67.1948                    1
…                  …                …                        …
1209596              1              62.8802                    1
1209597              1              62.8802                    1
1209598              1              62.8802                    1
1209599              1              62.8802                    1
1209600              1              62.6226                    1

         \TOTAL_CONS_REQUIRED_FLOW
0                             0.68
1                             0.68
2                             0.68
3                             0.68
4                             0.68
…                             …
1209596                       0.39
1209597                       0.39
1209598                       0.39
1209599                       0.39
1209600                       0.39

[1209601 rows x 130 columns]
```

[4]:
```
print(f"Data Share of WADI_attackdata")
print(test17_df.shape)
```

```
Data Share of WADI_attackdata
(172801, 130)
```

[6]:
```
print(f"Data Info of WADI_attackdata")
print(test17_df.info(verbose=True, show_counts=True))
print(test17_df)
```

```
Data Info of WADI_attackdata
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 172801 entries, 0 to 172800
Data columns (total 130 columns):
 #    Column                      Non-Null Count   Dtype
---   ------                      --------------   -----
 0    Row                         172801 non-null  int64
 1    Date                        172801 non-null  object
 2    Time                        172801 non-null  object
 3    \1_AIT_001_PV               172801 non-null  float64
 4    \1_AIT_002_PV               172801 non-null  float64
 5    \1_AIT_003_PV               172801 non-null  float64
 6    \1_AIT_004_PV               172801 non-null  float64
```

```
7    \1_AIT_005_PV            172801 non-null  float64
8    \1_FIT_001_PV            172801 non-null  float64
9    \1_LS_001_AL             172801 non-null  int64
10   \1_LS_002_AL             172801 non-null  int64
11   \1_LT_001_PV             172801 non-null  float64
12   \1_MV_001_STATUS         172801 non-null  int64
13   \1_MV_002_STATUS         172801 non-null  int64
14   \1_MV_003_STATUS         172801 non-null  int64
15   \1_MV_004_STATUS         172801 non-null  int64
16   \1_P_001_STATUS          172801 non-null  int64
17   \1_P_002_STATUS          172801 non-null  int64
18   \1_P_003_STATUS          172801 non-null  int64
19   \1_P_004_STATUS          172801 non-null  int64
20   \1_P_005_STATUS          172801 non-null  int64
21   \1_P_006_STATUS          172801 non-null  int64
22   \2_DPIT_001_PV           172801 non-null  float64
23   \2_FIC_101_CO            172801 non-null  float64
24   \2_FIC_101_PV            172801 non-null  float64
25   \2_FIC_101_SP            172801 non-null  float64
26   \2_FIC_201_CO            172801 non-null  float64
27   \2_FIC_201_PV            172801 non-null  float64
28   \2_FIC_201_SP            172801 non-null  float64
29   \2_FIC_301_CO            172801 non-null  float64
30   \2_FIC_301_PV            172801 non-null  float64
31   \2_FIC_301_SP            172801 non-null  float64
32   \2_FIC_401_CO            172801 non-null  float64
33   \2_FIC_401_PV            172801 non-null  float64
34   \2_FIC_401_SP            172801 non-null  float64
35   \2_FIC_501_CO            172801 non-null  float64
36   \2_FIC_501_PV            172801 non-null  float64
37   \2_FIC_501_SP            172801 non-null  float64
38   \2_FIC_601_CO            172801 non-null  float64
39   \2_FIC_601_PV            172801 non-null  float64
40   \2_FIC_601_SP            172801 non-null  float64
41   \2_FIT_001_PV            172801 non-null  float64
42   \2_FIT_002_PV            172801 non-null  float64
43   \2_FIT_003_PV            172801 non-null  float64
44   \2_FQ_101_PV             172801 non-null  float64
45   \2_FQ_201_PV             172801 non-null  float64
46   \2_FQ_301_PV             172801 non-null  float64
47   \2_FQ_401_PV             172801 non-null  float64
48   \2_FQ_501_PV             172801 non-null  float64
49   \2_FQ_601_PV             172801 non-null  float64
50   \2_LS_001_AL             0 non-null       float64
51   \2_LS_002_AL             0 non-null       float64
52   \2_LS_101_AH             172801 non-null  int64
53   \2_LS_101_AL             172801 non-null  int64
54   \2_LS_201_AH             172801 non-null  int64
```

```
55   \2_LS_201_AL             172801 non-null  int64
56   \2_LS_301_AH             172801 non-null  int64
57   \2_LS_301_AL             172801 non-null  int64
58   \2_LS_401_AH             172801 non-null  int64
59   \2_LS_401_AL             172801 non-null  int64
60   \2_LS_501_AH             172801 non-null  int64
61   \2_LS_501_AL             172801 non-null  int64
62   \2_LS_601_AH             172801 non-null  int64
63   \2_LS_601_AL             172801 non-null  int64
64   \2_LT_001_PV             172801 non-null  float64
65   \2_LT_002_PV             172801 non-null  float64
66   \2_MCV_007_CO            172801 non-null  int64
67   \2_MCV_101_CO            172801 non-null  float64
68   \2_MCV_201_CO            172801 non-null  float64
69   \2_MCV_301_CO            172801 non-null  float64
70   \2_MCV_401_CO            172801 non-null  float64
71   \2_MCV_501_CO            172801 non-null  float64
72   \2_MCV_601_CO            172801 non-null  float64
73   \2_MV_001_STATUS         172801 non-null  int64
74   \2_MV_002_STATUS         172801 non-null  int64
75   \2_MV_003_STATUS         172801 non-null  int64
76   \2_MV_004_STATUS         172801 non-null  int64
77   \2_MV_005_STATUS         172801 non-null  int64
78   \2_MV_006_STATUS         172801 non-null  int64
79   \2_MV_009_STATUS         172801 non-null  int64
80   \2_MV_101_STATUS         172801 non-null  int64
81   \2_MV_201_STATUS         172801 non-null  int64
82   \2_MV_301_STATUS         172801 non-null  int64
83   \2_MV_401_STATUS         172801 non-null  int64
84   \2_MV_501_STATUS         172801 non-null  int64
85   \2_MV_601_STATUS         172801 non-null  int64
86   \2_P_001_STATUS          0 non-null       float64
87   \2_P_002_STATUS          0 non-null       float64
88   \2_P_003_SPEED           172801 non-null  float64
89   \2_P_003_STATUS          172801 non-null  int64
90   \2_P_004_SPEED           172801 non-null  float64
91   \2_P_004_STATUS          172801 non-null  int64
92   \2_PIC_003_CO            172801 non-null  float64
93   \2_PIC_003_PV            172801 non-null  float64
94   \2_PIC_003_SP            172801 non-null  float64
95   \2_PIT_001_PV            172801 non-null  float64
96   \2_PIT_002_PV            172801 non-null  float64
97   \2_PIT_003_PV            172801 non-null  float64
98   \2_SV_101_STATUS         172801 non-null  int64
99   \2_SV_201_STATUS         172801 non-null  int64
100  \2_SV_301_STATUS         172801 non-null  int64
101  \2_SV_401_STATUS         172801 non-null  int64
102  \2_SV_501_STATUS         172801 non-null  int64
```

```
 103  \2_SV_601_STATUS           172801 non-null  int64
 104  \2A_AIT_001_PV             172801 non-null  float64
 105  \2A_AIT_002_PV             172801 non-null  float64
 106  \2A_AIT_003_PV             172801 non-null  float64
 107  \2A_AIT_004_PV             172801 non-null  float64
 108  \2B_AIT_001_PV             172801 non-null  float64
 109  \2B_AIT_002_PV             172801 non-null  float64
 110  \2B_AIT_003_PV             172801 non-null  float64
 111  \2B_AIT_004_PV             172801 non-null  float64
 112  \3_AIT_001_PV              172801 non-null  int64
 113  \3_AIT_002_PV              172801 non-null  float64
 114  \3_AIT_003_PV              172801 non-null  float64
 115  \3_AIT_004_PV              172801 non-null  float64
 116  \3_AIT_005_PV              172801 non-null  float64
 117  \3_FIT_001_PV              172801 non-null  float64
 118  \3_LS_001_AL               172801 non-null  int64
 119  \3_LT_001_PV               172801 non-null  float64
 120  \3_MV_001_STATUS           172801 non-null  int64
 121  \3_MV_002_STATUS           172801 non-null  int64
 122  \3_MV_003_STATUS           172801 non-null  int64
 123  \3_P_001_STATUS            172801 non-null  int64
 124  \3_P_002_STATUS            172801 non-null  int64
 125  \3_P_003_STATUS            172801 non-null  int64
 126  \3_P_004_STATUS            172801 non-null  int64
 127  \LEAK_DIFF_PRESSURE        172801 non-null  float64
 128  \PLANT_START_STOP_LOG      172801 non-null  int64
 129  \TOTAL_CONS_REQUIRED_FLOW  172801 non-null  float64
dtypes: float64(71), int64(57), object(2)
memory usage: 171.4+ MB
None
           Row        Date             Time  \1_AIT_001_PV  \1_AIT_002_PV  \
0            1   10/9/2017  6:00:00.000 PM        164.210       0.529486
1            2   10/9/2017  6:00:01.000 PM        164.210       0.529486
2            3   10/9/2017  6:00:02.000 PM        164.210       0.529486
3            4   10/9/2017  6:00:03.000 PM        164.210       0.529486
4            5   10/9/2017  6:00:04.000 PM        164.210       0.529486
…          …          …               …              …             …
172796  172797  10/11/2017  5:59:56.000 PM        172.959       0.547483
172797  172798  10/11/2017  5:59:57.000 PM        172.959       0.547483
172798  172799  10/11/2017  5:59:58.000 PM        172.915       0.583479
172799  172800  10/11/2017  5:59:59.000 PM        172.915       0.583479
172800  172801  10/11/2017  6:00:00.000 PM        172.915       0.583479

        \1_AIT_003_PV  \1_AIT_004_PV  \1_AIT_005_PV  \1_FIT_001_PV  \
0             11.9972        482.480       0.331167       0.001273
1             11.9972        482.480       0.331167       0.001273
2             11.9972        482.480       0.331167       0.001273
3             11.9972        482.480       0.331167       0.001273
```

```
4            11.9972        482.480       0.331167       0.001273
…              …            …             …              …
172796       11.9184        466.034       0.318217       0.001222
172797       11.9184        466.034       0.318217       0.001222
172798       11.9211        466.051       0.318317       0.001260
172799       11.9211        466.051       0.318317       0.001260
172800       11.9211        466.051       0.318317       0.001260

        \1_LS_001_AL  …  \3_MV_001_STATUS  \3_MV_002_STATUS  \
0                  0  …                 1                 1
1                  0  …                 1                 1
2                  0  …                 1                 1
3                  0  …                 1                 1
4                  0  …                 1                 1
…                …  …                 …                 …
172796             0  …                 1                 1
172797             0  …                 1                 1
172798             0  …                 1                 1
172799             0  …                 1                 1
172800             0  …                 1                 1

        \3_MV_003_STATUS  \3_P_001_STATUS  \3_P_002_STATUS  \3_P_003_STATUS  \
0                      1                 1                 1                 1
1                      1                 1                 1                 1
2                      1                 1                 1                 1
3                      1                 1                 1                 1
4                      1                 1                 1                 1
…                    …                 …                 …                 …
172796                 1                 1                 1                 1
172797                 1                 1                 1                 1
172798                 1                 1                 1                 1
172799                 1                 1                 1                 1
172800                 1                 1                 1                 1

        \3_P_004_STATUS  \LEAK_DIFF_PRESSURE  \PLANT_START_STOP_LOG  \
0                     1              62.6226                      1
1                     1              62.6226                      1
2                     1              62.6226                      1
3                     1              62.6226                      1
4                     1              62.6226                      1
…                   …                …                          …
172796                1              59.3546                      1
172797                1              59.3546                      1
172798                1              58.8102                      1
172799                1              58.8102                      1
172800                1              58.8102                      1

        \TOTAL_CONS_REQUIRED_FLOW
```

```
0                              0.39
1                              0.39
2                              0.39
3                              0.39
4                              0.39
…                              …
172796                         0.00
172797                         0.00
172798                         0.00
172799                         0.00
172800                         0.00

[172801 rows x 130 columns]
```

1.4 Classify data columns into Meta Group, Sensor Group or Actuator Group

[7]:
```
print(f"Classify the data columns into MetaGroup, Sensor Group and Actuator␣
 ↪Group - 14days")
# ========================================
# EXPLICIT COLUMN GROUPING
# ========================================
all_cols = train17_df.columns.tolist()

metadata_cols = []
actuator_ctrl_cols = []
sensor_cols = []

for col in all_cols:
    col_upper = col.upper()

    # Identify Metadata columns
    if any(k in col_upper for k in ["ROW", "DATE", "TIME"]):
        metadata_cols.append(col)

    # Identify Continuous Sensor Process Values (_PV and direct flow/pressure␣
 ↪metrics)
    elif "_PV" in col_upper or any(
        k in col_upper for k in ["LEAK_DIFF_PRESSURE",␣
 ↪"TOTAL_CONS_REQUIRED_FLOW"]
    ):
        sensor_cols.append(col)

    # Identify Actuators, Alarms, Controller Outputs, and Setpoints␣
 ↪(non-continuous sensors)
    elif any(
        k in col_upper
        for k in [
```

```
            "STATUS",
            "_AL",
            "_AH",
            "_CO",
            "_SP",
            "_SPEED",
            "PLANT_START_STOP_LOG",
        ]
    ):
        actuator_ctrl_cols.append(col)

    else:
        # Fallback safety: treat anything else as actuator/control if not␣
 ↪explicitly a sensor
        actuator_ctrl_cols.append(col)

print(f"\nGroup Breakdown:")
print(f" - Metadata Columns: {len(metadata_cols)}")
print(f" - Actuator Columns: {len(actuator_ctrl_cols)}")
print(f" - Sensor Columns: {len(sensor_cols)}")
```

```
Classify the data columns into MetaGroup, Sensor Group and Actuator Group -
14days

Group Breakdown:
 - Metadata Columns: 3
 - Actuator Columns: 81
 - Sensor Columns: 46
```

[8]:
```
print(f"Classify the data columns into MetaGroup, Sensor Group and Actuator␣
 ↪Group - attackdata")
# ========================================
# EXPLICIT COLUMN GROUPING
# ========================================
tall_cols = test17_df.columns.tolist()

tmetadata_cols = []
tactuator_ctrl_cols = []
tsensor_cols = []

for col in tall_cols:
    col_upper = col.upper()

    # Identify Metadata columns
    if any(k in col_upper for k in ["ROW", "DATE", "TIME"]):
        tmetadata_cols.append(col)
```

```
    # Identify Continuous Sensor Process Values (_PV and direct flow/pressure␣
↪metrics)
    elif "_PV" in col_upper or any(
        k in col_upper for k in ["LEAK_DIFF_PRESSURE",␣
↪"TOTAL_CONS_REQUIRED_FLOW"]
    ):
        tsensor_cols.append(col)

    # Identify Actuators, Alarms, Controller Outputs, and Setpoints␣
↪(non-continuous sensors)
    elif any(
        k in col_upper
        for k in [
            "STATUS",
            "_AL",
            "_AH",
            "_CO",
            "_SP",
            "_SPEED",
            "PLANT_START_STOP_LOG",
        ]
    ):
        tactuator_ctrl_cols.append(col)

    else:
        # Fallback safety: treat anything else as actuator/control if not␣
↪explicitly a sensor
        tactuator_ctrl_cols.append(col)

print(f"\nGroup Breakdown:")
print(f" - Metadata Columns: {len(tmetadata_cols)}")
print(f" - Actuator Columns: {len(tactuator_ctrl_cols)}")
print(f" - Sensor Columns: {len(tsensor_cols)}")
```

```
Classify the data columns into MetaGroup, Sensor Group and Actuator Group -
attackdata

Group Breakdown:
 - Metadata Columns: 3
 - Actuator Columns: 81
 - Sensor Columns: 46
```

1.5 Add Attack Lablel to Attack data

[9]:
```
print(f"Add Attack Label and Classify the attack label in WADI_attackdata")
# Clean column names in case of whitespace artifacts
test17_df.columns = test17_df.columns.str.strip()

# ========================================
# PARSE TIMESTAMP USING 'Date' AND 'Time' COLUMNS
# ========================================
# Create a temporary datetime series directly from 'DATE' and 'TIME'
# without adding a permanent 'DateTime' column to test17_df
temp_datetime = pd.to_datetime(
    test17_df["Date"].astype(str).str.strip()
    + " "
    + test17_df["Time"].astype(str).str.strip(),
    format='mixed',
    errors="coerce"
)

# ========================================
# 3. DEFINE ATTACK TIME RANGES
# ========================================
attack_ranges = [
    ("09/10/2017 19:25:00", "09/10/2017 19:50:16"),
    ("10/10/2017 10:24:10", "10/10/2017 10:34:00"),
    ("10/10/2017 10:55:00", "10/10/2017 11:24:00"),
    ("10/10/2017 11:07:46", "10/10/2017 11:12:15"),
    ("10/10/2017 11:30:40", "10/10/2017 11:44:50"),
    ("10/10/2017 13:39:30", "10/10/2017 13:50:40"),
    ("10/10/2017 14:48:17", "10/10/2017 14:59:55"),
    ("10/10/2017 14:53:44", "10/10/2017 15:00:32"),
    ("10/10/2017 17:40:00", "10/10/2017 17:49:40"),
    ("11/10/2017 10:55:00", "11/10/2017 10:56:27"),
    ("11/10/2017 11:17:54", "11/10/2017 11:31:20"),
    ("11/10/2017 11:36:31", "11/10/2017 11:47:00"),
    ("11/10/2017 11:59:00", "11/10/2017 12:05:00"),
    ("11/10/2017 12:07:30", "11/10/2017 12:10:52"),
    ("11/10/2017 12:16:00", "11/10/2017 12:25:36"),
    ("11/10/2017 15:26:30", "11/10/2017 15:37:00"),
]

# ========================================
# 4. INITIALIZE AND POPULATE ATTACK LABELS
# ========================================
# Initialize attack label column to 1 as default
test17_df["ATTACK_LABEL"] = 1

# Iterate through each attack range and mark matching rows as -1
```

```
for start_str, end_str in attack_ranges:
    start_dt = pd.to_datetime(start_str, format="%d/%m/%Y %H:%M:%S")
    end_dt = pd.to_datetime(end_str, format="%d/%m/%Y %H:%M:%S")

    mask = (temp_datetime >= start_dt) & (temp_datetime <= end_dt)
    test17_df.loc[mask, "ATTACK_LABEL"] = -1

# ========================================
# 5. VERIFICATION
# ========================================
print("\nLabel distribution summary:")
print(test17_df["ATTACK_LABEL"].value_counts())
print(f"\nFinal DataFrame shape: {test17_df.shape}")
print(f"Last column name: {test17_df.columns[-1]}")
```

```
Add Attack Label and Classify the attack label in WADI_attackdata

Label distribution summary:
ATTACK_LABEL
 1    162816
-1      9985
Name: count, dtype: int64

Final DataFrame shape: (172801, 131)
Last column name: ATTACK_LABEL
```

1.6 Task 3: Generate the Synthetic Data from selected dataset

[10]:
```
def run_synthetic_generation_cli():
    print("=" * 60)
    print("TASK 3: SYNTHETIC DATASET GENERATION")
    print("=" * 60)


    # ========================================
    # INPUT 1: DATASET SELECTION
    # ========================================
    while True:
        ds_input = input("Select dataset [1: train17_df, 2: test17_df] (Enter 1␣
 ↪or 2): ").strip()
        if ds_input == "1":
            raw_df = train17_df.copy()
            file_name = "WADI_14days.csv"
            break
        elif ds_input == "2":
            raw_df = test17_df.copy()
            file_name = "WADI_attackdata.csv"
```

```
            break
        else:
            print("Invalid selection. Please enter '1' or '2'.")

    print(f"-> Selected Dataset: ({file_name})")

    # ========================================
    # INPUT 2: PERCENTAGE OF ROWS TO RANDOMLY SAMPLE (1 - 100%) - Control the␣
↪processing time
    # ========================================
    while True:
        try:
            row_percentage = int(input("Enter percentage of sample rows to␣
↪select from dataset (Integer range: 1 - 100%): ").strip())
            if 1 <= row_percentage <= 100:
                break
            else:
                print("Error: Percentage must be between 1 and 100.")
        except ValueError:
            print("Error: Invalid input. Please enter a valid integer.")

    print(f"-> Selected Row Sub-sample Percentage: {row_percentage}%")

    # Randomly select sample rows as selected_sample_df
    n_sample_rows = int(len(raw_df) * (row_percentage / 100.0))
    selected_sample_df = raw_df.sample(n=n_sample_rows, random_state=42).
↪reset_index(drop=True)
    print(f"-> 'selected_sample_df' created with {len(selected_sample_df)} rows␣
↪out of {len(raw_df)}.")

    # ========================================
    # INPUT 3: NORMALIZATION SCALER SELECTION
    # ========================================
    while True:
        scaler_input = input("Select scaler type [1: StandardScaler, 2:␣
↪MinMaxScaler] (Enter 1 or 2): ").strip()
        if scaler_input == "1":
            scaler_type = "standard"
            break
        elif scaler_input == "2":
            scaler_type = "minmax"
            break
        else:
            print("Invalid selection. Please enter '1' or '2'.")

    print(f"-> Selected Scaler: {scaler_type.upper()}")
```

```
    # ========================================
    # INPUT 4: SYNTHETIC GENERATION PERCENTAGE (1 - 1000%)
    # ========================================
    while True:
        try:
            percentage = int(input("Enter synthetic sample generation␣
↪percentage (Integer range: 1 - 1000%): ").strip())
            if 1 <= percentage <= 1000:
                break
            else:
                print("Error: Percentage must be between 1 and 1000.")
        except ValueError:
            print("Error: Invalid input. Please enter a valid integer.")

    print(f"-> Selected Synthetic Percentage: {percentage}%")

    # ========================================
    # INPUT 5: K NEIGHBORS (2 - 5)
    # ========================================
    while True:
        try:
            k = int(input("Enter number of nearest neighbors k (Integer range:␣
↪2 - 5): ").strip())
            if 2 <= k <= 5:
                break
            else:
                print("Error: k must be between 2 and 5.")
        except ValueError:
            print("Error: Invalid input. Please enter a valid integer.")

    print(f"-> Selected k: {k}")

    sensor_data = selected_sample_df[sensor_cols].ffill().bfill().fillna(0).
↪to_numpy(dtype=float)

    # Start timestamp and time
    generation_start_time = datetime.now()
    print(f"-> Generation Start Timestamp: {generation_start_time.
↪strftime('%Y-%m-%d %H:%M:%S')}")

    # ========================================
    # CUSTOM NORMALIZATION FROM SCRATCH
    # ========================================
    if scaler_type == "standard":
        means = np.mean(sensor_data, axis=0)
        stds = np.std(sensor_data, axis=0)
        stds[stds == 0] = 1.0
```

```
        normalized_sensors = (sensor_data - means) / stds
    else:
        mins = np.min(sensor_data, axis=0)
        maxs = np.max(sensor_data, axis=0)
        ranges = maxs - mins
        ranges[ranges == 0] = 1.0
        normalized_sensors = (sensor_data - mins) / ranges

    working_df = selected_sample_df.copy()
    working_df[sensor_cols] = normalized_sensors

    # ========================================
    # Compute and Select K-NN
    # ========================================
    print("Computing k-nearest neighbors")
    n_samples = len(normalized_sensors)
    k_neighbors = np.empty((n_samples, k), dtype=int)

    for i in range(n_samples):
        dists = np.sum(np.abs(normalized_sensors - normalized_sensors[i]),␣
↪axis=1)
        dists[i] = np.inf
        k_neighbors[i] = np.argsort(dists)[:k]

        if (i + 1) % 20000 == 0 or (i + 1) == n_samples:
            print(f"Processed {i + 1} / {n_samples} rows...")

    print("Finished k-NN computation successfully.")

    # ========================================
    # SEQUENTIAL SYNTHETIC SAMPLE GENERATION
    # ========================================
    n_real_samples = len(working_df)
    n_synthetic_samples = int(n_real_samples * (percentage / 100.0))
    chunk_size = 10000

    print(f"Generating {n_synthetic_samples} synthetic samples sequentially␣
↪with chunk_size={chunk_size}...")

    synthetic_rows = []
    generated_count = 0

    while generated_count < n_synthetic_samples:
        current_chunk_size = min(chunk_size, n_synthetic_samples -␣
↪generated_count)

        for _ in range(current_chunk_size):
```

```
            idx = random.randint(0, n_real_samples - 1)
            base_sample = working_df.iloc[idx]

            neighbor_idx = random.choice(k_neighbors[idx])
            neighbor_sample = working_df.iloc[neighbor_idx]

            alpha = random.random()
            synthetic_row = base_sample.copy()
            synthetic_row[sensor_cols] = base_sample[sensor_cols] + alpha *␣
↪(neighbor_sample[sensor_cols] - base_sample[sensor_cols])
            synthetic_rows.append(synthetic_row)

        generated_count += current_chunk_size
        print(f"Generated {generated_count} / {n_synthetic_samples} synthetic␣
↪samples...")

    synthetic_data_df = pd.DataFrame(synthetic_rows)

    # ========================================
    # POPULATE ROW, DATE, AND TIME COLUMNS
    # ========================================
    # 1. Row column starts from integer 1 and sequentially increases by 1
    synthetic_data_df["Row"] = range(1, len(synthetic_data_df) + 1)

    # RESET DF INDEX
    synthetic_data_df = synthetic_data_df.reset_index(drop=True)

    # 2 & 3. Date starts at "10/10/2017" (DD/MM/YYYY) and increases daily;
    # Time starts at "00:00:00" (HH:MM:SS) and increases by 1 second per row.
    base_datetime = datetime(2017, 10, 10, 0, 0, 0)
    datetimes = [base_datetime + timedelta(seconds=i) for i in␣
↪range(len(synthetic_data_df))]

    synthetic_data_df["Date"] = [dt.strftime("%d/%m/%Y") for dt in datetimes]
    synthetic_data_df["Time"] = [dt.strftime("%H:%M:%S") for dt in datetimes]

    # Reorder columns to place Row, Date, Time at the front if they exist or␣
↪were created
    cols_order = ["Row", "Date", "Time"] + [c for c in synthetic_data_df.
↪columns if c not in ["Row", "Date", "Time"]]
    synthetic_data_df = synthetic_data_df[cols_order]


    # ========================================
    # SAVE TO PROCESSED DIRECTORY
    # ========================================
    output_dir = Path.home() / "Projects" / "P1" / "data" / "processed"
```

```
    output_dir.mkdir(parents=True, exist_ok=True)

    output_file = output_dir / "synthetic_data.csv"
    synthetic_data_df.to_csv(output_file, index=False)

    # Track overall end timestamp and calculate duration
    generation_end_time = datetime.now()
    duration_minutes = (generation_end_time - generation_start_time).
↪total_seconds() / 60.0

    print("\n" + "=" * 60)
    print("SUCCESSFULLY GENERATED AND SAVED SYNTHETIC DATASET")
    print("=" * 60)
    print(f"Start Timestamp : {generation_start_time.strftime('%Y-%m-%d %H:%M:
↪%S')}")
    print(f"End Timestamp   : {generation_end_time.strftime('%Y-%m-%d %H:%M:
↪%S')}")
    print(f"Duration        : {duration_minutes:.4f} minutes")
    print(f"Output File Path: {output_file}")
    print(f"Synthetic Dataset Shape: {synthetic_data_df.shape}")

    return synthetic_data_df
```

[11]:
```
# To run the interactive CLI program, uncomment the line below:
synthetic_data_df = run_synthetic_generation_cli()
```

```
============================================================
TASK 3: SYNTHETIC DATASET GENERATION
============================================================

Select dataset [1: train17_df, 2: test17_df] (Enter 1 or 2):  2

-> Selected Dataset: (WADI_attackdata.csv)

Enter percentage of sample rows to select from dataset (Integer range: 1 -
100%):  10

-> Selected Row Sub-sample Percentage: 10%
-> 'selected_sample_df' created with 17280 rows out of 172801.

Select scaler type [1: StandardScaler, 2: MinMaxScaler] (Enter 1 or 2):  1

-> Selected Scaler: STANDARD

Enter synthetic sample generation percentage (Integer range: 1 - 1000%):  100

-> Selected Synthetic Percentage: 100%

Enter number of nearest neighbors k (Integer range: 2 - 5):  2

-> Selected k: 2
-> Generation Start Timestamp: 2026-10-26 23:15:42
```

```
Computing k-nearest neighbors
Processed 17280 / 17280 rows…
Finished k-NN computation successfully.
Generating 17280 synthetic samples sequentially with chunk_size=10000…
Generated 10000 / 17280 synthetic samples…
Generated 17280 / 17280 synthetic samples…

============================================================
SUCCESSFULLY GENERATED AND SAVED SYNTHETIC DATASET
============================================================
Start Timestamp : 2026-10-26 23:15:42
End Timestamp   : 2026-10-26 23:20:19
Duration        : 4.6104 minutes
Output File Path: /home/ringo/Projects/P1/data/processed/synthetic_data.csv
Synthetic Dataset Shape: (17280, 131)
```

[12]:
```
print('Data info of synthetic_data_df - generated')
print(synthetic_data_df.info(verbose=True,show_counts=True))
print(synthetic_data_df)
```

```
Data info of synthetic_data_df - generated
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 17280 entries, 0 to 17279
Data columns (total 131 columns):
 #    Column                      Non-Null Count  Dtype
---   ------                      --------------  -----
 0    Row                         17280 non-null  int64
 1    Date                        17280 non-null  object
 2    Time                        17280 non-null  object
 3    \1_AIT_001_PV               17280 non-null  float64
 4    \1_AIT_002_PV               17280 non-null  float64
 5    \1_AIT_003_PV               17280 non-null  float64
 6    \1_AIT_004_PV               17280 non-null  float64
 7    \1_AIT_005_PV               17280 non-null  float64
 8    \1_FIT_001_PV               17280 non-null  float64
 9    \1_LS_001_AL                17280 non-null  int64
 10   \1_LS_002_AL                17280 non-null  int64
 11   \1_LT_001_PV                17280 non-null  float64
 12   \1_MV_001_STATUS            17280 non-null  int64
 13   \1_MV_002_STATUS            17280 non-null  int64
 14   \1_MV_003_STATUS            17280 non-null  int64
 15   \1_MV_004_STATUS            17280 non-null  int64
 16   \1_P_001_STATUS             17280 non-null  int64
 17   \1_P_002_STATUS             17280 non-null  int64
 18   \1_P_003_STATUS             17280 non-null  int64
 19   \1_P_004_STATUS             17280 non-null  int64
 20   \1_P_005_STATUS             17280 non-null  int64
 21   \1_P_006_STATUS             17280 non-null  int64
```

```
22   \2_DPIT_001_PV           17280 non-null  float64
23   \2_FIC_101_CO            17280 non-null  float64
24   \2_FIC_101_PV            17280 non-null  float64
25   \2_FIC_101_SP            17280 non-null  float64
26   \2_FIC_201_CO            17280 non-null  float64
27   \2_FIC_201_PV            17280 non-null  float64
28   \2_FIC_201_SP            17280 non-null  float64
29   \2_FIC_301_CO            17280 non-null  float64
30   \2_FIC_301_PV            17280 non-null  float64
31   \2_FIC_301_SP            17280 non-null  float64
32   \2_FIC_401_CO            17280 non-null  float64
33   \2_FIC_401_PV            17280 non-null  float64
34   \2_FIC_401_SP            17280 non-null  float64
35   \2_FIC_501_CO            17280 non-null  float64
36   \2_FIC_501_PV            17280 non-null  float64
37   \2_FIC_501_SP            17280 non-null  float64
38   \2_FIC_601_CO            17280 non-null  float64
39   \2_FIC_601_PV            17280 non-null  float64
40   \2_FIC_601_SP            17280 non-null  float64
41   \2_FIT_001_PV            17280 non-null  float64
42   \2_FIT_002_PV            17280 non-null  float64
43   \2_FIT_003_PV            17280 non-null  float64
44   \2_FQ_101_PV             17280 non-null  float64
45   \2_FQ_201_PV             17280 non-null  float64
46   \2_FQ_301_PV             17280 non-null  float64
47   \2_FQ_401_PV             17280 non-null  float64
48   \2_FQ_501_PV             17280 non-null  float64
49   \2_FQ_601_PV             17280 non-null  float64
50   \2_LS_001_AL             0 non-null      float64
51   \2_LS_002_AL             0 non-null      float64
52   \2_LS_101_AH             17280 non-null  int64
53   \2_LS_101_AL             17280 non-null  int64
54   \2_LS_201_AH             17280 non-null  int64
55   \2_LS_201_AL             17280 non-null  int64
56   \2_LS_301_AH             17280 non-null  int64
57   \2_LS_301_AL             17280 non-null  int64
58   \2_LS_401_AH             17280 non-null  int64
59   \2_LS_401_AL             17280 non-null  int64
60   \2_LS_501_AH             17280 non-null  int64
61   \2_LS_501_AL             17280 non-null  int64
62   \2_LS_601_AH             17280 non-null  int64
63   \2_LS_601_AL             17280 non-null  int64
64   \2_LT_001_PV             17280 non-null  float64
65   \2_LT_002_PV             17280 non-null  float64
66   \2_MCV_007_CO            17280 non-null  int64
67   \2_MCV_101_CO            17280 non-null  float64
68   \2_MCV_201_CO            17280 non-null  float64
69   \2_MCV_301_CO            17280 non-null  float64
```

```
70   \2_MCV_401_CO             17280 non-null  float64
71   \2_MCV_501_CO             17280 non-null  float64
72   \2_MCV_601_CO             17280 non-null  float64
73   \2_MV_001_STATUS          17280 non-null  int64
74   \2_MV_002_STATUS          17280 non-null  int64
75   \2_MV_003_STATUS          17280 non-null  int64
76   \2_MV_004_STATUS          17280 non-null  int64
77   \2_MV_005_STATUS          17280 non-null  int64
78   \2_MV_006_STATUS          17280 non-null  int64
79   \2_MV_009_STATUS          17280 non-null  int64
80   \2_MV_101_STATUS          17280 non-null  int64
81   \2_MV_201_STATUS          17280 non-null  int64
82   \2_MV_301_STATUS          17280 non-null  int64
83   \2_MV_401_STATUS          17280 non-null  int64
84   \2_MV_501_STATUS          17280 non-null  int64
85   \2_MV_601_STATUS          17280 non-null  int64
86   \2_P_001_STATUS           0 non-null      float64
87   \2_P_002_STATUS           0 non-null      float64
88   \2_P_003_SPEED            17280 non-null  float64
89   \2_P_003_STATUS           17280 non-null  int64
90   \2_P_004_SPEED            17280 non-null  float64
91   \2_P_004_STATUS           17280 non-null  int64
92   \2_PIC_003_CO             17280 non-null  float64
93   \2_PIC_003_PV             17280 non-null  float64
94   \2_PIC_003_SP             17280 non-null  float64
95   \2_PIT_001_PV             17280 non-null  float64
96   \2_PIT_002_PV             17280 non-null  float64
97   \2_PIT_003_PV             17280 non-null  float64
98   \2_SV_101_STATUS          17280 non-null  int64
99   \2_SV_201_STATUS          17280 non-null  int64
100  \2_SV_301_STATUS          17280 non-null  int64
101  \2_SV_401_STATUS          17280 non-null  int64
102  \2_SV_501_STATUS          17280 non-null  int64
103  \2_SV_601_STATUS          17280 non-null  int64
104  \2A_AIT_001_PV            17280 non-null  float64
105  \2A_AIT_002_PV            17280 non-null  float64
106  \2A_AIT_003_PV            17280 non-null  float64
107  \2A_AIT_004_PV            17280 non-null  float64
108  \2B_AIT_001_PV            17280 non-null  float64
109  \2B_AIT_002_PV            17280 non-null  float64
110  \2B_AIT_003_PV            17280 non-null  float64
111  \2B_AIT_004_PV            17280 non-null  float64
112  \3_AIT_001_PV             17280 non-null  float64
113  \3_AIT_002_PV             17280 non-null  float64
114  \3_AIT_003_PV             17280 non-null  float64
115  \3_AIT_004_PV             17280 non-null  float64
116  \3_AIT_005_PV             17280 non-null  float64
117  \3_FIT_001_PV             17280 non-null  float64
```

```
 118  \3_LS_001_AL               17280 non-null  int64
 119  \3_LT_001_PV               17280 non-null  float64
 120  \3_MV_001_STATUS           17280 non-null  int64
 121  \3_MV_002_STATUS           17280 non-null  int64
 122  \3_MV_003_STATUS           17280 non-null  int64
 123  \3_P_001_STATUS            17280 non-null  int64
 124  \3_P_002_STATUS            17280 non-null  int64
 125  \3_P_003_STATUS            17280 non-null  int64
 126  \3_P_004_STATUS            17280 non-null  int64
 127  \LEAK_DIFF_PRESSURE        17280 non-null  float64
 128  \PLANT_START_STOP_LOG      17280 non-null  int64
 129  \TOTAL_CONS_REQUIRED_FLOW  17280 non-null  float64
 130  ATTACK_LABEL               17280 non-null  int64
dtypes: float64(72), int64(57), object(2)
memory usage: 17.3+ MB
None
         Row        Date      Time  \1_AIT_001_PV  \1_AIT_002_PV  \
0          1  10/10/2017  00:00:00      -0.278839      -0.100859
1          2  10/10/2017  00:00:01       0.019601      -0.281455
2          3  10/10/2017  00:00:02      -0.311427      -0.022629
3          4  10/10/2017  00:00:03      -0.336274      -0.070739
4          5  10/10/2017  00:00:04       0.317781       0.132600
…      …         …        …              …              …
17275  17276  10/10/2017  04:47:55       0.259715       0.112578
17276  17277  10/10/2017  04:47:56       0.176525      -0.166534
17277  17278  10/10/2017  04:47:57      -0.458855      -0.139621
17278  17279  10/10/2017  04:47:58       0.210846      -0.084441
17279  17280  10/10/2017  04:47:59       0.208309       0.128996

       \1_AIT_003_PV  \1_AIT_004_PV  \1_AIT_005_PV  \1_FIT_001_PV  \
0           0.388730      -0.659473      -0.871300       1.528610
1          -0.021834       1.375742       1.677275      -0.631556
2          -0.067482      -1.474081      -1.044623      -0.631809
3           0.077461       0.898707       0.834073      -0.631856
4           0.276231      -0.026853      -0.009214       1.586457
…              …              …              …              …
17275       0.245339      -0.120630      -0.525846      -0.631871
17276       0.014259       1.172152       1.164498      -0.631610
17277       0.234515      -0.766691      -0.997692      -0.028376
17278      -0.408131       1.450367       1.851667      -0.631834
17279      -0.327878       0.229511       0.653278      -0.631714

       \1_LS_001_AL  …  \3_MV_002_STATUS  \3_MV_003_STATUS  \3_P_001_STATUS  \
0                 0  …                 1                 1                1
1                 0  …                 1                 1                1
2                 0  …                 1                 1                1
3                 0  …                 1                 1                1
4                 0  …                 1                 1                1
```

```
…              …  …                 …                 …                 …
17275            0  …                 1                 1                 1
17276            0  …                 1                 1                 1
17277            0  …                 1                 1                 1
17278            0  …                 1                 1                 1
17279            0  …                 1                 1                 1

       \3_P_002_STATUS  \3_P_003_STATUS  \3_P_004_STATUS  \LEAK_DIFF_PRESSURE  \
0                    1                 1                 1             0.211357
1                    1                 1                 1             4.565072
2                    1                 1                 1             0.755823
3                    1                 1                 1            -0.271648
4                    1                 1                 1             0.217997
…                  …                 …                 …                 …
17275                1                 1                 1            -0.153197
17276                1                 1                 1            -0.480508
17277                1                 1                 1             8.376353
17278                1                 1                 1            -0.527284
17279                1                 1                 1            -0.027919

       \PLANT_START_STOP_LOG  \TOTAL_CONS_REQUIRED_FLOW  ATTACK_LABEL
0                          1                   0.285854             1
1                          1                  -0.104867             1
2                          1                   1.653380             1
3                          1                  -1.211911             1
4                          1                   0.394388             1
…                        …                          …             …
17275                      1                  -0.473882             1
17276                      1                  -1.211911             1
17277                      1                  -0.148280             1
17278                      1                  -1.211911             1
17279                      1                   0.177321             1

[17280 rows x 131 columns]
```

[ ]: